# "RFC 3514: The Security Flag" for C++

**Document #:**
P3514R0

**Date:**
2025-04-01

**Audience:**
LEWGI, WG21

**Reply-to:**
Steve Downey <sdowney@gmail.com>

**Source:**
https://github.com/steve-downey/wg21org

evil-bit.org

heads/main-0-g41ab8b7

## Table of Contents

**7 References**

Abstract: Compilers, interpreters, static analysis tools, and the like often have difficulty distinguishing between code that has malicious intent and code that is merely unusual. We define a security flag trait as a means of distinguishing the two cases. Prior art is established by (Bellovin 2003).

# 1. Introduction

Detecting and determining malicious intent is a difficult problem. To solve this, we require that code with malicious intent provide a specialization of a variable template that can be checked by a concept. Benign types will specialize the boolean variable template to false, which will also be the default, malicious types will specialize the template to be true.

The semantic constraint of *malicious intent* means that code that is merely incorrect will be unaffected. Compilers may not infer malicious intent nor may they optimize based on such an inference. Compilers may not, on their own, create time travel paradoxes or put the existence of the timeline at risk.

Optimization based on the assertion of evil intent is, however, an open research question. Implementations SHOULD take care that programs containing evil code aer still translated correctly.

> The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

## 2. Comparison Table

| Before | After |
| --- | --- |
| `template<typename T>`<br>`int meow(T t);` | `template<typename T>`<br>`int meow(T t) requires !std::security::evil<T>;` |

## 3. Implementation

```
namespace security {
template< class T >
constexpr bool evil_bit = false;

template< class T >
concept evil = evil_bit<std::remove_cvref_t<T>>;
}
```

Note that as the evil bit is not a member of `T` the evil bit can be set externally to the type, in order support the marking of types that were implemented before the evil bit was standardized.

Malicious types that do not define `evil_bit<T>` to be true are IFNDR.

## 4. Experience

The widespread adoption and improvements in Internet safety and security since RFC 3514 was proposed in 2003 speak for themselves.

## 5. Wording

Based on (“N4917: Working Draft, Standard for Programming Language c++”)

### 5.1. Add to [meta.type.synop]

```
namespace std::security {

  template<class T>
    constexpr bool evil_bit = false;

  template<class T>
    concept evil = /*see below*/;
}
```

### 5.2. Add a new section [meta.security.evil]

```
template<class T>
  concept evil = evil_bit<remove_cvref_t<T>>;
```

Given a variable t of type T, T models `evil` if and only if the intent of the type is malicious.

```
template<class>
  constexpr bool evil_bit = false;
```

Remarks: Pursuant to `[namespace.std]`, users may specialize `evil_bit` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (`[expr.const]`) and have type `const bool`.

### 5.3. Feature-test macro [version.syn]

Add the following macro definition to [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_evil_bit 20XXXXL
```

## 6. Open Questions

- Should `evil_bit` be set for deprecated facilities?

- Should non-Unicode locales be marked `evil`?
- Should `std::regex`?

The proposed facility may have interactions with the in-flight proposals for profiles. (Reis 2025; Voutilainen 2025; Jabot 2025; Gill et al. 2024; Stroustrup 2024c, 2024b, 2024a; Laverdière, Lapkowski, and Gros 2025; Sutter 2025) *et al.*

This proposal places no requirements on any profiles paper and can be adopted independently of any proposal in this area.

Similarly, although it would be desirable for contract checks to not be `evil`, this proposal places no requirements on mandating the lack of evil in contracts. *Caveat lector.*

## 7. References


Bellovin, Steven. 2003. “The Security Flag in the IPv4 Header.” Request for Comments. RFC 3514; RFC Editor. https://doi.org/10.17487/RFC3514.

Bradner, Scott O. 1997. “Key words for use in RFCs to Indicate Requirement Levels.” Request for Comments. RFC 2119; RFC Editor. https://doi.org/10.17487/RFC2119.

Gill, Mungo, Corentin Jabot, John Lakos, Joshua Berne, and Timur Doumler. 2024. “P3543R0: Response to Core Safety Profiles (P3081).” https://wg21.link/p3543r0; WG21.

Jabot, Corentin. 2025. “P3586R0: The Plethora of Problems with Profiles.” https://wg21.link/p3586r0; WG21.

Köppe, Thomas. 2022. “N4917: Working Draft, Standard for Programming Language c++.” https://wg21.link/n4917; WG21.

Laverdière, Marc-André, Christopher Lapkowski, and Charles-Henri Gros. 2025. “P3402R2: A Safety Profile Verifying Initialization.” https://wg21.link/p3402r2; WG21.

Leiba, Barry. 2017. “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words.” Request for Comments. RFC 8174; RFC Editor. https://doi.org/10.17487/RFC8174.

Reis, Gabriel Dos. 2025. “P3589R1: C++ Profiles: The Framework.” https://wg21.link/p3589r1; WG21.

Stroustrup, Bjarne. 2024a. “P3274R0: A Framework for Profiles Development.” https://wg21.link/p3274r0; WG21.

———. 2024b. “P3446R0: Profile Invalidation - Eliminating Dangling Pointers.” https://wg21.link/p3446r0; WG21.

———. 2024c. “P3447R0: Profiles Syntax.” https://wg21.link/p3447r0; WG21.

Sutter, Herb. 2025. “P3081R2: Core Safety Profiles for c++26.” https://wg21.link/p3081r2; WG21.

Voutilainen, Ville. 2025. “P3608R0: Contracts and Profiles: What Can We Reasonably Ship in c++26.” https://wg21.link/p3608r0; WG21.

Exported: 2025-04-06 22:49:58

https://github.com/steve-downey/wg21org/blob/41ab8b7d308b07152bbdeb24d0b6f8f989a53210/evil-bit.org